The batch/streaming *continuum*

Every Spark job we wrote in Unit 2 had the same shape. Read a fixed input, build a DAG, execute it stage by stage, write a fixed output, exit. The input was complete before we started, which is exactly what let Catalyst plan the whole computation up front and let us reason about the job as a finite thing with a runtime. This style of computation is **batch processing**: the dataset is bounded, the job runs to completion, and the answer it produces is final for that input.

Now consider what the Statcast pipeline looks like from the other end. During a game, the tracking system in the stadium emits a fully measured event a few seconds after every pitch. Those events do not arrive as a tidy seven-million-row Parquet file. They arrive one at a time, in real time, all season long, from fifteen stadiums at once. The dataset has no end. There is no moment at which the input is complete, because tomorrow there are more games. Computation over data like this is **stream processing**: the input is unbounded, the job runs indefinitely, and every answer it produces is provisional, correct only as of the data seen so far.

It is a continuum, not a switch

The textbook framing makes batch and streaming sound like two different kinds of system. The more useful framing is that they are two ends of a single axis, and the axis is **freshness**: how stale are you willing to let the answer be? Suppose the team's analytics department wants pitcher fatigue indicators. If the coaching staff reviews them weekly, a weekly batch job is the right tool. If they review them every morning, a nightly batch job is the right tool. If the pitching coach wants them between innings, no batch cadence is fast enough, and we are in streaming territory. Same data, same computation, different freshness requirement.

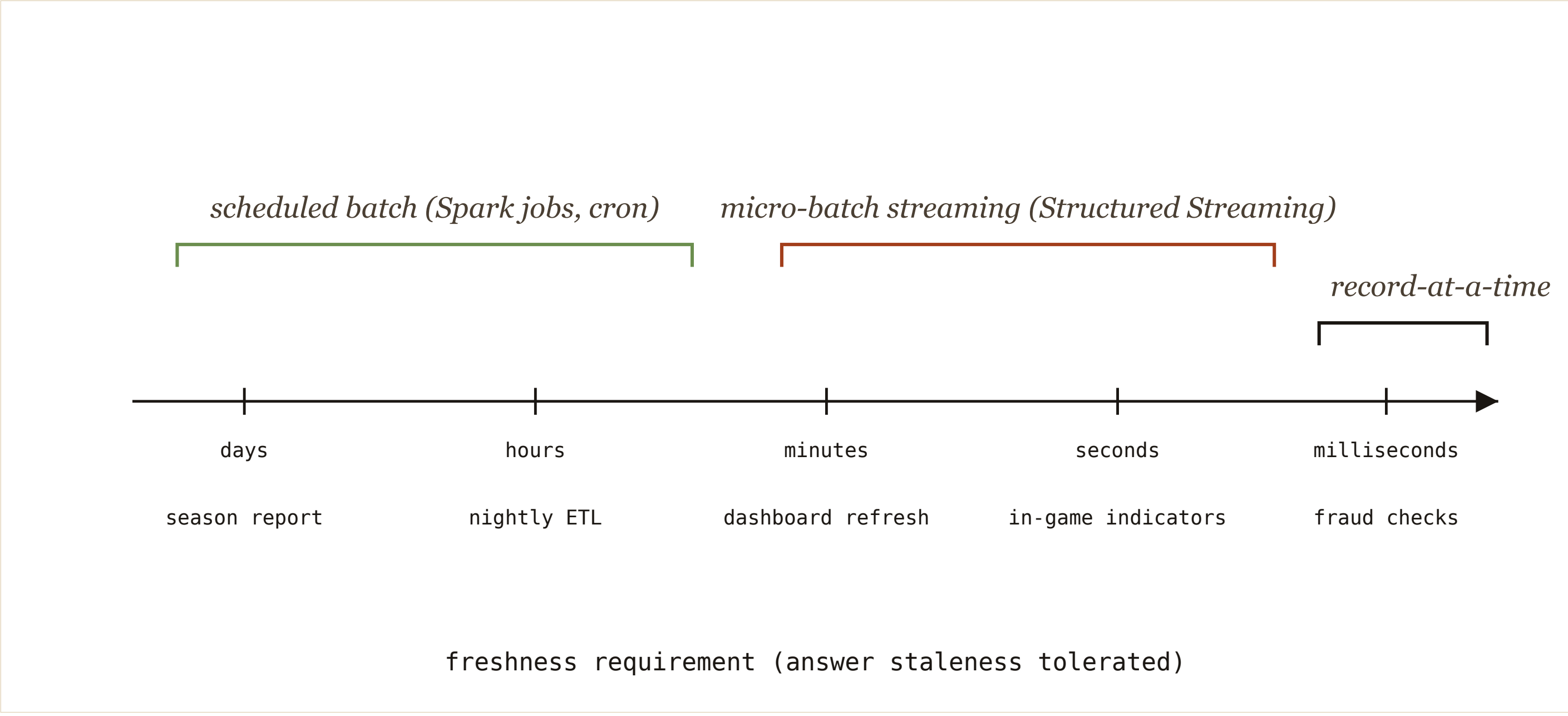
scheduled batch (Spark jobs, cron)
micro-batch streaming (Structured Streaming)
record-at-a-time
days
hours
minutes
seconds
milliseconds
season report
nightly ETL
dashboard refresh
in-game indicators
fraud checks
freshness requirement (answer staleness tolerated)

FIG 3.1 *The freshness axis. Batch and streaming are not two kinds of system; they are regions of one continuum, and the workload's staleness tolerance tells you where to stand.*

Reading the continuum left to right also reads as a cost curve. Every step toward fresher answers costs more: more always-on infrastructure, more operational complexity, more careful failure handling. A senior data scientist's first question about any "we need real-time" request should be the freshness question, asked with numbers. A surprising fraction of real-time requests turn out to be satisfied by a five-minute micro-batch, which is an order of magnitude cheaper to build and operate than a millisecond pipeline.

It is worth being concrete about why the right end of the axis costs what it does, because the cost is mostly not hardware. A batch job that fails is an inconvenience: the input files are still sitting in the bucket, untouched, and you rerun the job after lunch. Failure recovery is "run it again," and the data waits patiently for you. A streaming job that fails is an incident: the input keeps arriving whether or not anyone is processing it, the backlog grows by the minute, downstream consumers are watching a frozen dashboard, and whoever owns the pipeline gets called at 0300. The data does not wait. Unit 1 taught us that at cluster scale, failure is the weather; streaming adds that the weather now happens to a system that is never allowed to stop. Most of this unit's machinery (durable logs, checkpoints, replayable offsets, exactly-once commits) exists to make that 0300 call survivable, and the machinery has to be designed in from the start, because it cannot be bolted onto a pipeline after the fact.

Why not just rerun the batch job?

Here is the obvious first design for fresh answers, and it is worth taking seriously before rejecting it. We already have a Spark batch job that computes, say, per-pitcher whiff rate for the season. Why not run it every sixty seconds? Each run reads the full season-to-date data and overwrites the output table. Always fresh, no new concepts.

Sounds great, right? But watch what happens to the cost. In April the job reads a few hundred thousand rows and finishes in seconds. By September it reads seven million rows every sixty seconds to incorporate the few dozen pitches that arrived since the last run. The work per run grows with the total history, while the new information per run stays constant. We are paying full price to recompute an answer that is 99.999 percent identical to the one we just threw away. The fix is the central computational idea of this unit: **incremental computation**, in which the system maintains the answer and updates it using only the new data, carrying forward whatever intermediate information it needs between updates. That carried-forward information has a name, **state**, and managing it correctly is where most of the difficulty in this unit lives.

Micro-batch vs record-at-a-time

Granting that computation should be incremental, there is still a choice about the unit of increment, and it splits the streaming world into two camps.

A **micro-batch** system collects arriving records for a short interval (hundreds of milliseconds to minutes), then runs a small batch job over just that slice, updating state as it goes. Spark Structured Streaming works this way, and it allows all of our Unit 2 knowledge to transfer almost intact: each micro-batch is planned by Catalyst, executed in stages, and visible in the Spark UI, exactly like the jobs we have been reading all unit. The price is a latency floor. The answer can never be fresher than the batch interval plus the processing time.

A **record-at-a-time** system (Apache Flink is the standard example) pushes each event through a standing pipeline of operators the moment it arrives, with no batching boundary at all. Latency drops to milliseconds. The price is paid elsewhere: per-record processing has more overhead per unit of throughput, and the failure-recovery story has to work without batch boundaries to lean on, which makes it considerably more intricate.

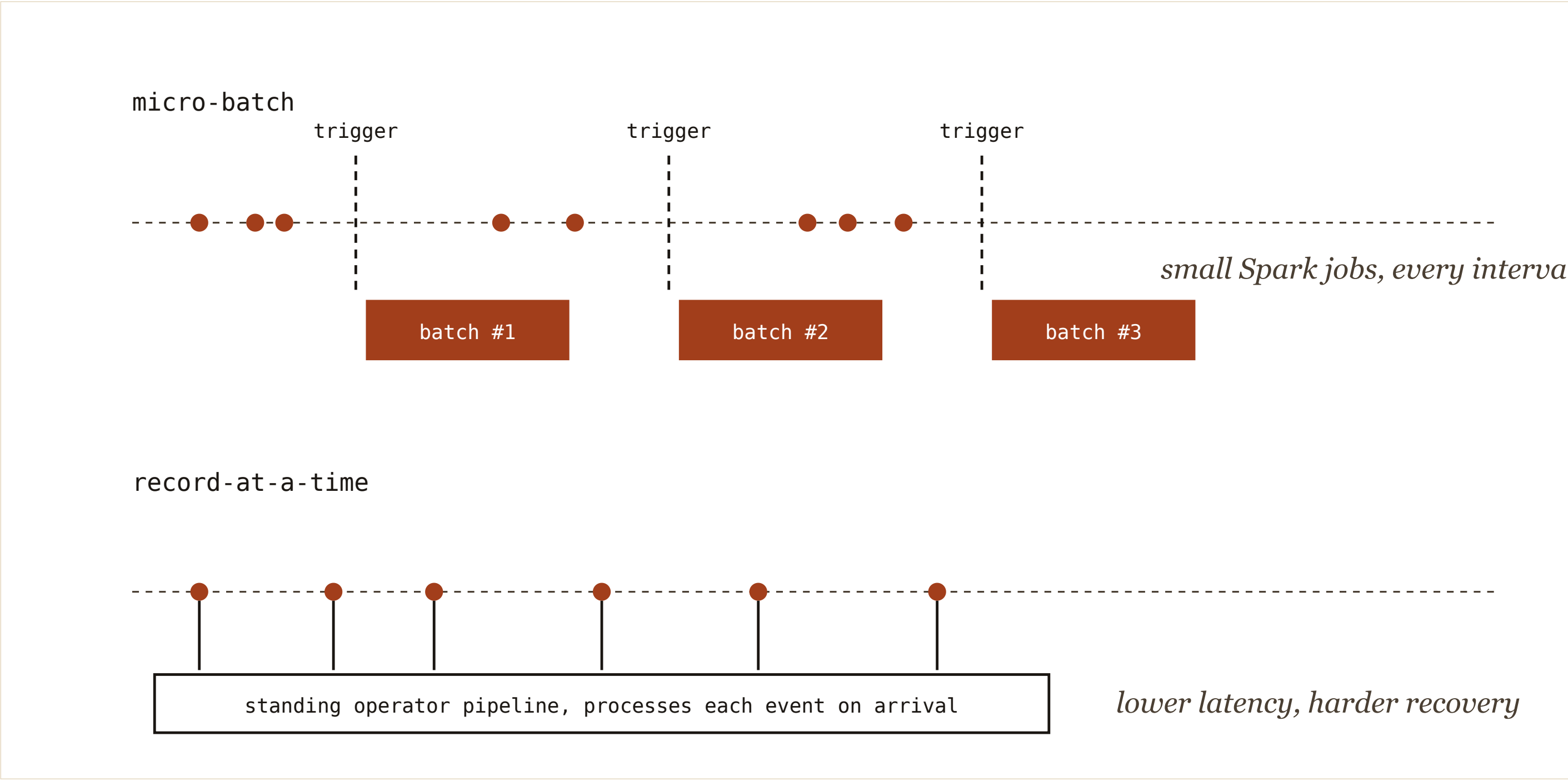
micro-batch
trigger
trigger
trigger
small Spark jobs, every interval
batch #1
batch #2
batch #3
record-at-a-time
standing operator pipeline, processes each event on arrival
lower latency, harder recovery

FIG 3.2 *Two units of increment. Micro-batch systems chop the stream at trigger boundaries and run a small batch job per slice; record-at-a-time systems push every event through a standing pipeline as it arrives.*

We will commit to the micro-batch side, for the same reason we previously committed to Spark: the concepts transfer. A student who can reason about state, time, and failure in Structured Streaming can pick up Flink's versions of the same ideas in a week. The reverse path is harder, because Flink's machinery hides the batch heritage that makes the model legible.

Before any of that machinery can run, though, there is a logistics problem we have quietly skipped. The stadium emits events. Our cluster computes on events. What sits between them? In batch land the answer was object storage, and files in a bucket are a terrible interface for fifteen stadiums writing concurrently while several consumers read at different speeds. We need a piece of infrastructure purpose-built for moving unbounded event flows from many producers to many consumers, durably, in order.

Kafka and *the log*

We ended last time with a logistics problem. Many producers emit events continuously; many consumers want to read them, each at its own pace, each starting from its own position, each able to crash and come back. Lets make the problem concrete before we look at the system that solves it.

MIDN A runs the ingestion side of our hypothetical live Statcast pipeline. Fifteen stadium tracking systems push pitch events to her. MIDN B runs three consumers: a live dashboard that wants events within seconds, an hourly job that writes Parquet to the lake, and an alerting job that watches for velocity drops. Notice the shape of the trouble. The producers and consumers run at different speeds. The dashboard is fast; the Parquet writer works in hourly gulps. If MIDN B's alerting job crashes for twenty minutes, the events from those twenty minutes must not vanish; the job needs to come back and resume where it stopped. And when next week a fourth consumer appears, it should be able to plug in without MIDN A changing anything on the producer side.

Direct connections fail every requirement at once. Producers would have to track which consumers exist, buffer for the slow ones, and replay for the crashed ones, which makes every producer responsible for every consumer's problems. What we want in the middle is a buffer that is durable, shared, and ordered. SD212's concurrency unit gave us the single-machine version of this idea: the producer/consumer queue that decouples threads running at different speeds. Kafka is that queue grown up, distributed across machines, and made durable. But it differs from a queue in one decision so consequential it deserves its own section.

The log

A queue hands each message to a consumer and then deletes it. Kafka instead keeps every event in a **log**: an append-only sequence of records on disk, each identified by its position. Producers append at the tail. Consumers read at whatever position they like, and reading deletes nothing. Each record's position in the log is its **offset**, a monotonically increasing integer that names that record forever (or at least until retention expires, which we get to shortly).

You have met this data structure before, wearing a different uniform. SD321 showed us that a database's durability story rests on its write-ahead log: every change is appended to a sequential log before it touches the tables, and recovery means replaying the log. Kafka takes that internal organ and makes it the entire organism. The log is not an implementation detail behind the API; the log *is* the API.

The consequences of read-without-delete do a lot of work for MIDN B. Her three consumers read the same log independently, each tracking its own offset. The fast dashboard reads near the tail; the hourly Parquet writer trails behind; neither knows the other exists. When the alerting job crashes, its offset simply stops advancing, and on restart it resumes from the last offset it recorded. And the fourth consumer next week starts at whatever offset it likes, including offset zero, replaying history that was written before the consumer existed. Replayability turns out to be the property the whole unit leans on; hold onto it.

Events, not states

The log also changes what we store, not just how. SD321's tables hold current state: the row for a player holds his team *now*, and a trade is an UPDATE that destroys the old value in place. A log holds **events**: immutable records of things that happened, each stamped with when it happened, never modified after the fact. "Player traded from team X to team Y at time T" goes into the log and stays there; the player's current team is something you *derive* by reading the events in order. The relationship between the two views is worth saying precisely, because it organizes a lot of modern data architecture: current state is a fold over the event history. Given the log you can always reconstruct the table, at any point in time you like; given only the table, the history is gone and no computation brings it back.

This is why the events in our pipeline should be designed as facts, not as commands or as snapshots of mutable rows. A pitch event records what the tracking system measured, immutably; if the stadium later issues a correction, the correction is itself a new event, and consumers decide how to apply it. Designing the event schema (what fields, what timestamp, what key) is a real engineering artifact in any streaming system, and in the lab you will read GitHub Archive's event schema with exactly this critical eye: every event carries its type, its actor, its repository, and crucially its `created_at`, the event-time stamp that the second half of this unit revolves around.

Topics and partitions

One log is not enough, for two reasons. Different event kinds should not share a sequence (pitch events and, say, lineup-change events are different streams), and a single log lives on a single machine, which caps throughput at one machine's disk and network. Kafka's answers are topics and partitions. A **topic** is a named category of events, like `statcast.pitches`. Each topic is split into some number of **partitions**, and each partition is its own independent append-only log with its own offset sequence, placed on its own broker so that the topic's total throughput scales out across machines. This is the same word, and genuinely the same move, as the data partitioning we did in Unit 1: split the thing across machines so no single machine is the bottleneck.

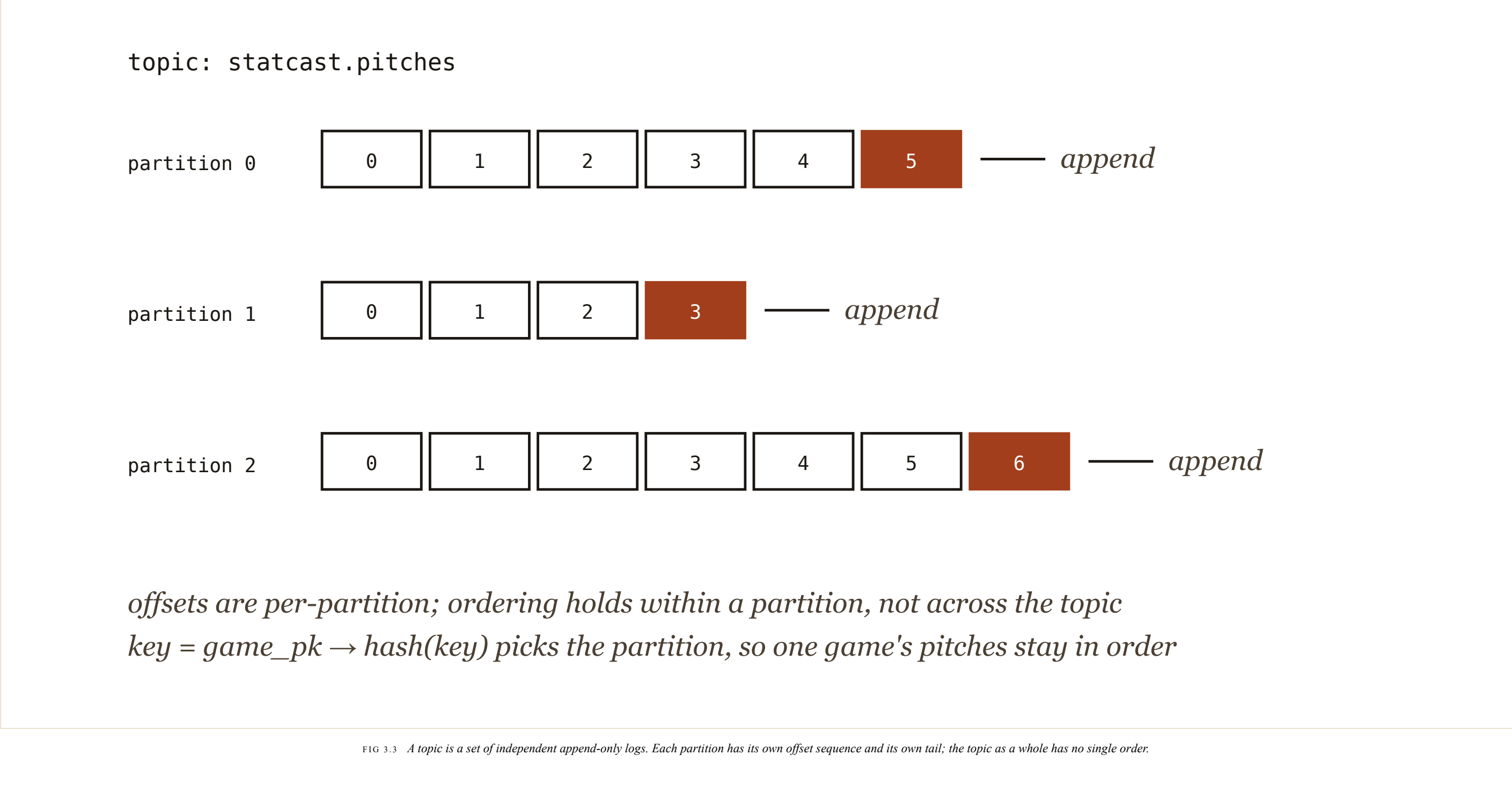
topic: statcast.pitches
partition 0
0
1
2
3
4
5
append
partition 1
0
1
2
3
append
partition 2
0
1
2
3
4
5
6
append
offsets are per-partition; ordering holds within a partition, not across the topic
key = game_pk → hash(key) picks the partition, so one game's pitches stay in order

FIG 3.3 *A topic is a set of independent append-only logs. Each partition has its own offset sequence and its own tail; the topic as a whole has no single order.*

Partitioning buys throughput and costs ordering. Kafka guarantees order *within* a partition and promises nothing *across* partitions. If pitch 47 of a game lands in partition 0 and pitch 48 lands in partition 2, a consumer may see them in either order. The control knob is the record key. When a producer attaches a key to a record, Kafka routes the record by hashing the key to pick a partition, the same hash-to-bucket move as the hash tables of SD311 and the shuffle routing of Unit 2. Records with equal keys land in the same partition and are therefore totally ordered with respect to each other. For our pipeline the right key is `game_pk`: every pitch of a given game shares a key, lands in one partition, and arrives at consumers in game order, while different games spread across partitions for throughput. Choosing the key is a real design decision, and a skewed one (imagine keying by `home_team` during a Yankees doubleheader) recreates exactly the skew pathology we have diagnosed, this time in the transport layer.

Brokers and replication

The machines that store partitions and serve clients are **brokers**. A production Kafka deployment is a cluster of brokers with partitions spread among them. And here Unit 1 hands us the next question by reflex: a partition lives on a broker, brokers are machines, and failure is the weather. What happens when a broker dies?

The answer is the answer it always is: replication. Each partition is copied to a configurable number of brokers, replication factor 3 being the convention for the same reasons it was the HDFS convention. One replica is the **leader**, which handles all produces and consumes for that partition; the others are **followers**, which replicate the leader's log and stand by. The followers that are fully caught up form the **in-sync replica set**, and when a leader's broker dies, one of the in-sync followers is promoted to leader, usually within seconds and invisibly to well-behaved clients. There is a producer-side knob worth knowing called `acks`: a producer can consider a record sent when the leader has it (`acks=1`, faster, can lose the record if the leader dies before followers copy it) or when the full in-sync set has it (`acks=all`, slower, durable). You should hear PACELC rumbling under that sentence. It is the Unit 1 latency-versus-consistency tradeoff, surfacing per record.

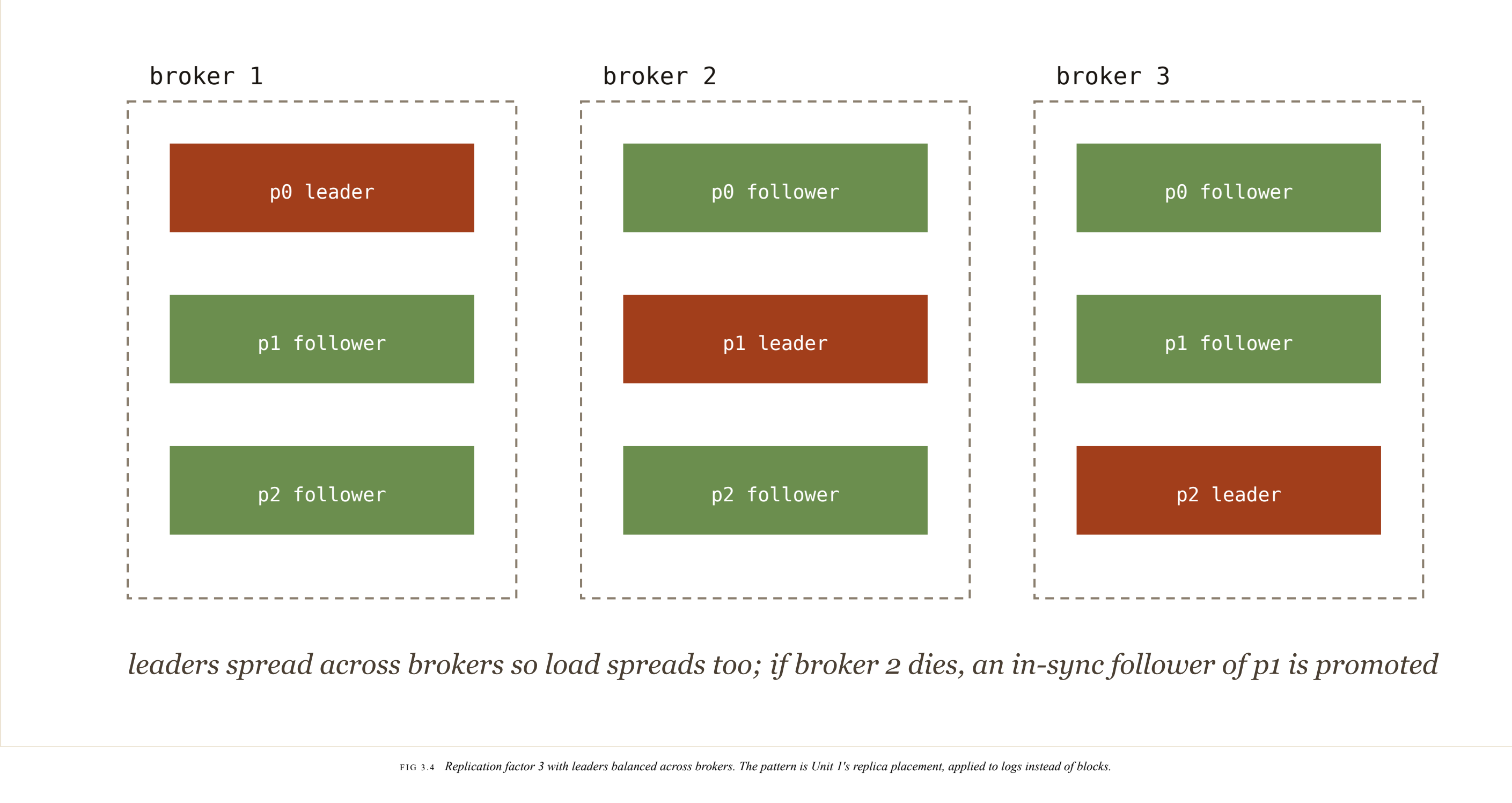

FIG 3.4 *Replication factor 3 with leaders balanced across brokers. The pattern is Unit 1's replica placement, applied to logs instead of blocks.*

Consumer groups

One consumer reading three partitions is easy. The interesting case is the opposite: a consumer that cannot keep up alone and needs to become several processes sharing the work. Kafka's unit of shared consumption is the **consumer group**. Consumers that declare the same group id divide the topic's partitions among themselves, each partition going to exactly one member of the group, and the group collectively tracks one set of offsets. If a member crashes, its partitions are reassigned to the survivors (a **rebalance**); if members are added, partitions redistribute to spread load. Scaling a consumer is therefore as simple as starting another copy of it with the same group id, up to a hard ceiling: a group with more members than the topic has partitions leaves the extras idle, which is why partition count is the first capacity decision made when a topic is created and an annoying one to change later.

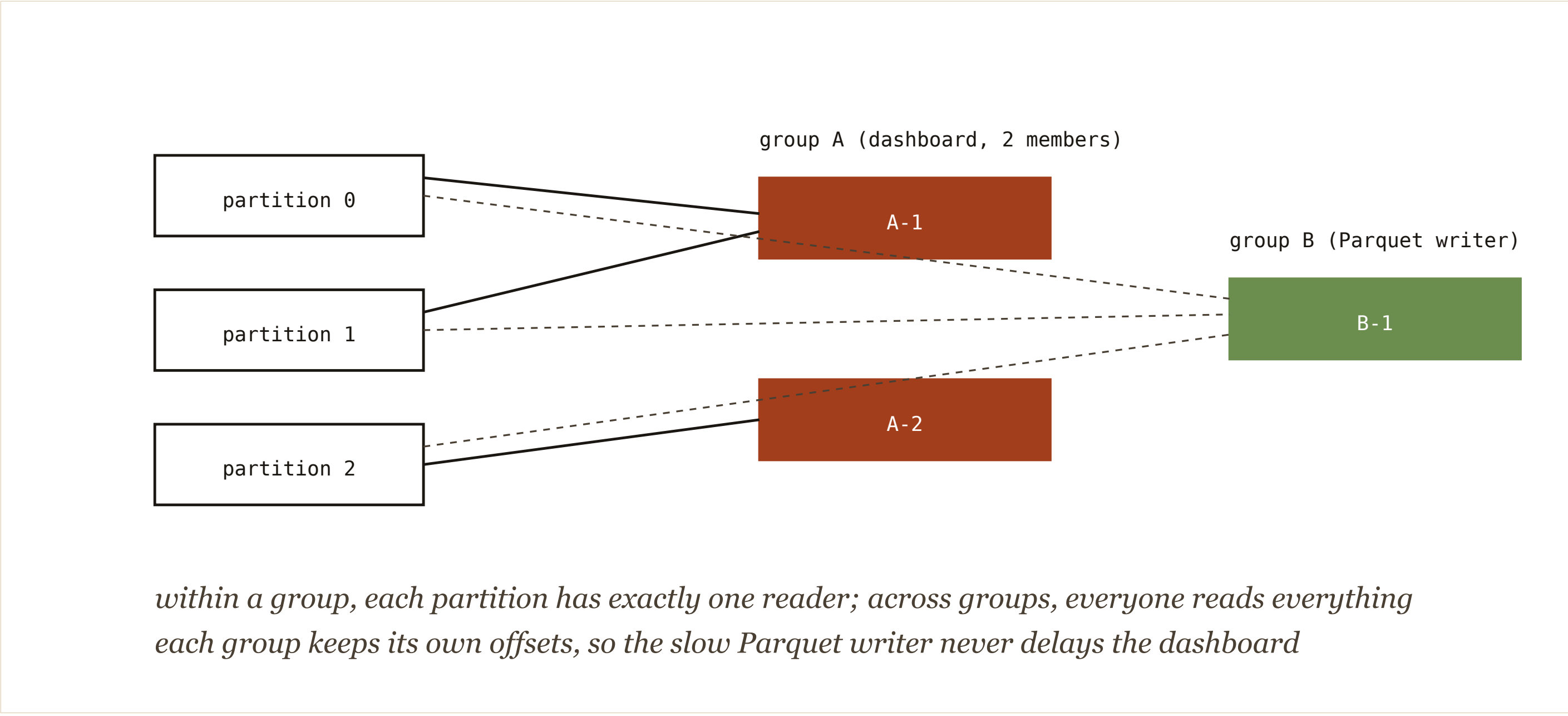
group A (dashboard, 2 members)
partition 0
partition 1
partition 2
A-1
A-2
group B (Parquet writer)
B-1
within a group, each partition has exactly one reader; across groups, everyone reads everything
each group keeps its own offsets, so the slow Parquet writer never delays the dashboard

FIG 3.5 *Two consumer groups on one topic. Group A splits the partitions between two members; group B reads the whole topic alone, at its own pace, with its own offsets.*

Retention, and what Kafka is not

The log cannot grow forever on finite disks, so each topic carries a **retention** policy, typically time-based: records older than, say, seven days are deleted from the head of the log regardless of who has or hasn't read them. Inside the retention window, the log is a replayable history; outside it, the data is gone, and any consumer that falls more than a retention window behind has permanently lost records. Sizing retention is a real decision with a real bill attached, because retention times throughput is disk.

It is just as important to be clear about what Kafka does not do. Kafka moves and stores events; it computes nothing. It will not aggregate, join, window, or filter for us (we set aside Kafka's own stream-processing add-ons, which we will not use). In lab you will stand the whole thing up with Docker Compose, produce the GitHub Archive event stream into a topic, and consume it, precisely so you can feel that boundary: data flowing beautifully, and no answers anywhere. To get answers, a computation engine has to subscribe to the log and maintain results incrementally as events arrive. We own such an engine.

The Structured *Streaming* model

We have an engine that plans and executes queries over bounded DataFrames, and we have a log full of events that never ends. Spark's bridge between them, Structured Streaming, is built on one idea, and the idea is good enough that once you see it, half the API explains itself.

The unbounded table

Treat the stream as a table that grows. Every event arriving on the Kafka topic is conceptually a new row appended to an **unbounded table**, an input table with a fixed schema and no last row. A streaming computation is then just a query against that table, written in exactly the DataFrame API, and the result of the query is itself a table that updates as the input grows. Nothing about the query says "streaming." `groupBy("pitcher").count()` means what it has always meant; what changes is that the table underneath it is still filling in.

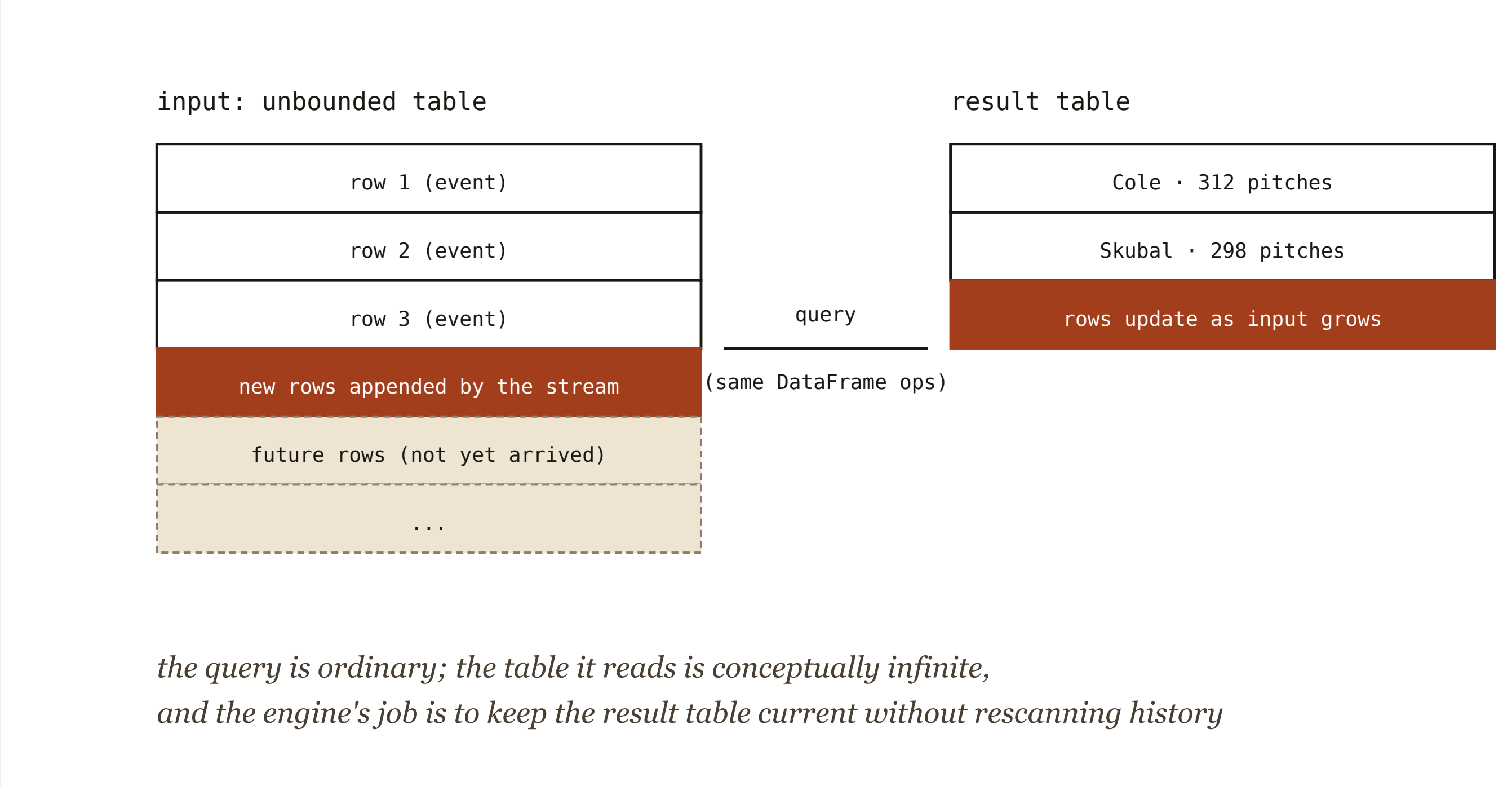
input: unbounded table
row 1 (event)
row 2 (event)
row 3 (event)
new rows appended by the stream
future rows (not yet arrived)
...
query
(same DataFrame ops)
result table
Cole · 312 pitches
Skubal · 298 pitches
rows update as input grows
the query is ordinary; the table it reads is conceptually infinite,
and the engine's job is to keep the result table current without rescanning history

FIG 3.6 *The unbounded table model. A stream is a table with no last row; a streaming query is an ordinary query whose result is kept up to date as rows arrive.*

The model is a contract, not an implementation. No system could actually rescan an infinite table on every update; that is the rerun-the-batch-job design we costed out las class and rejected. The contract says: the result table must at all times equal what you *would* get by running the query over all input seen so far. The engine's job is to honor that contract incrementally.

The micro-batch loop

Here is how it honors it. A Structured Streaming query runs as a loop driven by a **trigger**, the policy that decides when the next micro-batch fires (as fast as possible, on a fixed interval like `Trigger.ProcessingTime("30 seconds")`, or once and stop). Each cycle of the loop does the same five things.

1. Ask the source what is new. For Kafka this is an offset interval per partition: "last batch ended at offset 4,107; the log is now at 4,933."
2. Plan an incremental job over just those records. Catalyst plans it; it is a genuinely ordinary Spark job over a small bounded slice.
3. Combine the slice with carried-forward state. For a running count per pitcher, the state is the counts so far; the batch's job is to add this slice's contribution.
4. Write the changed results to the sink, in the requested output mode.
5. Commit progress: record the offset interval and the new state version to the checkpoint, then go around again.

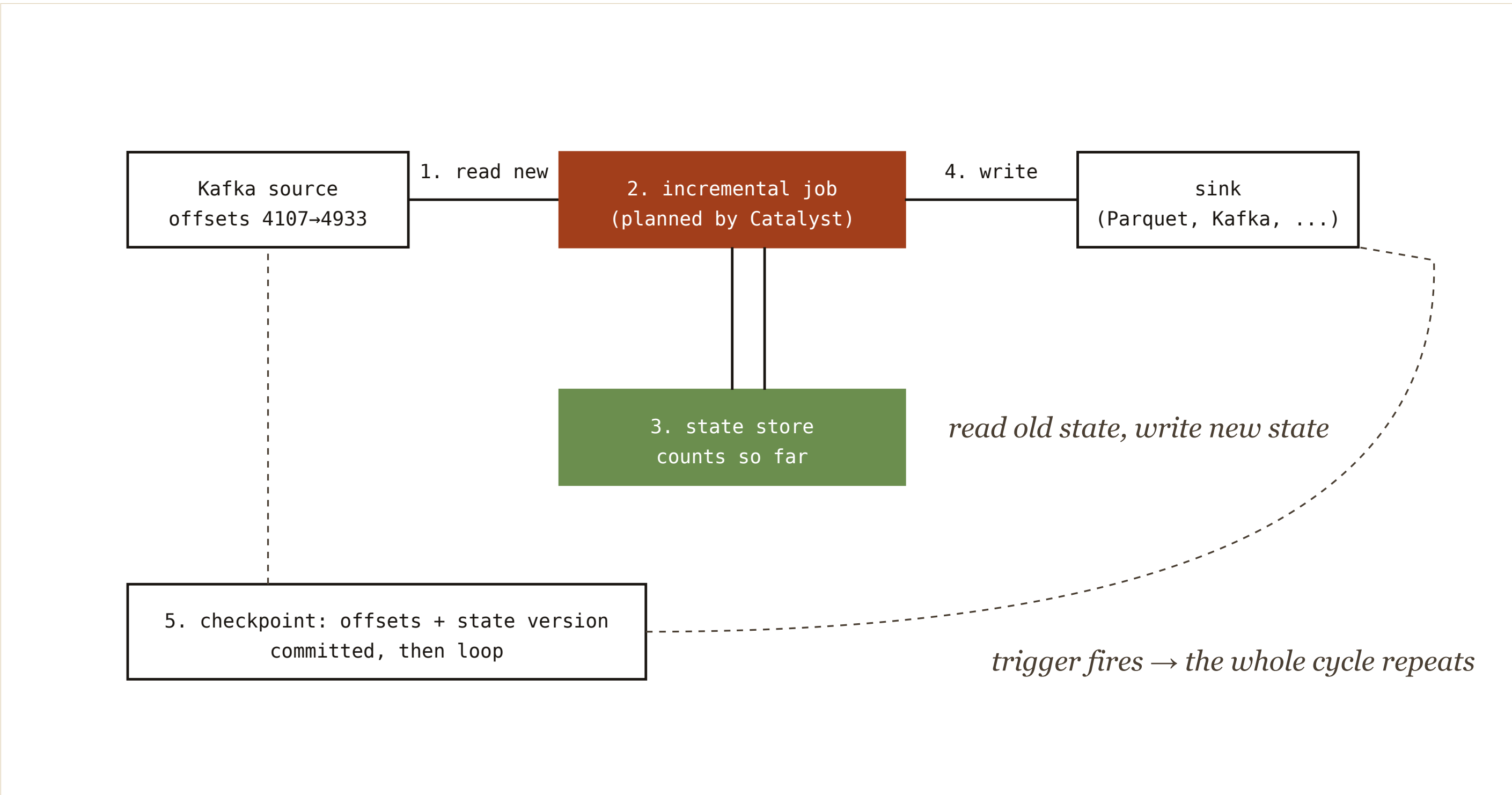

FIG 3.7 *One cycle of the micro-batch loop. Each cycle is a small, ordinary Spark job plus two pieces of bookkeeping: state carried between cycles, and a checkpoint recording how far we have gotten.*

Step 3 is where the new difficulty lives, so lets watch it happen with real numbers. MIDN A's query counts pitches per pitcher with a thirty-second trigger. Batch 17 reads offsets 4,107 through 4,932: 826 pitch events, of which 41 are Skubal's. The incremental job groups the slice, producing partial counts (Skubal: 41), then looks up the carried state (Skubal: 1,204 entering the batch), adds, writes the new state (Skubal: 1,245), and emits the updated row to the sink. Batch 18 fires thirty seconds later with 790 new events and repeats the dance from where batch 17 left off. At no point did anything reread the 1,204 historical pitches; the state *is* the compressed memory of them. The contract from Fig 3.6 is honored (the result equals what a full rescan would say) at the cost of a few kilobytes of carried counts instead of a full-history scan, which is the entire economic argument for streaming in one example.

Now lets be precise about which queries need this machinery at all. A stateless transformation (filter the pitches, parse the JSON, project columns) needs nothing carried between batches; each slice is processed and forgotten, and such pipelines are nearly as easy to operate as batch jobs. The moment the query aggregates, the result for a group depends on every batch that ever contributed to it, and the engine must keep **state**: the intermediate values (counts, sums, distinct sets) that let it update results without revisiting history. State is kept partitioned by grouping key across the executors, exactly as the shuffle would distribute it, and it persists from batch to batch. State is also a liability with a growth rate. A per-pitcher count grows slowly and harmlessly; a per-game aggregation grows all season unless something eventually retires finished games; group by something unbounded like a per-pitch UUID and state grows without limit until the executors fall over. "What is the state, what is its key, and what bounds its size" is the first question to ask of any streaming query, and the watermark lecture to come is, among other things, the answer to "what bounds it."

Output modes and sinks

One subtlety the batch world never made us face: when results update continuously, what exactly should be written out each cycle? Structured Streaming makes the choice explicit as the **output mode**. *Append* emits only rows that are final and will never change, the natural mode for stateless pipelines. *Update* emits the rows whose values changed this batch, the natural mode for live aggregations feeding a dashboard. *Complete* rewrites the entire result table every batch, tolerable only when the result is small. Not every combination of query and mode is legal (an aggregation cannot run in append mode until something can declare its groups finished, which, again, waits on watermarks), and the engine will refuse the illegal ones at startup rather than guess.

The destination itself, the **sink**, is the last leg: Parquet files on object storage, another Kafka topic, a database via foreachBatch. Sinks reenter the story in a serious way in the final lecture, because the guarantee a pipeline can honestly claim end to end turns out to hinge on what the sink can promise about duplicate writes.

What this looks like in code

```
events = (spark.readStream
    .format("kafka")
    .option("kafka.bootstrap.servers", "kafka:9092")
    .option("subscribe", "statcast.pitches")
    .load()
    .select(from_json(col("value").cast("string"), pitch_schema).alias("p"))
    .select("p.*"))

counts = events.groupBy("pitcher").count()

query = (counts.writeStream
    .outputMode("update")
    .format("console")
    .option("checkpointLocation", "s3a://sd411/chk/pitcher_counts")
    .trigger(processingTime="30 seconds")
    .start())
```

Read it against the loop in Fig 3.7 and notice how little is new. The middle line is Unit 2 verbatim. The novelties are the bookends: `readStream` instead of `read`, and a `writeStream` that names a trigger, an output mode, and a checkpoint location. That checkpoint location is mandatory for a reason. The query never finishes, so it will eventually be killed (deploy, crash, power event), and the checkpoint's offsets-plus-state record is precisely what lets it restart and resume as though nothing happened. Lineage, which rescued lost partitions for us, cannot rescue a computation whose inputs stretch back weeks; nobody replays a season to recover a count. The checkpoint replaces replay-from-the-beginning with resume-from-last-commit.

So we can keep a result table current over an endless input. Now look back at what we actually computed: pitches per pitcher, over all time. Almost no real question is "over all time." The dashboard wants pitches per pitcher *in the last ten minutes*; the alerting job wants average velocity *per inning*; the trending report wants pageviews *per hour*. Real streaming questions are nearly always questions about time, and the moment time enters the query, we have to confront an uncomfortable fact we have been stepping around: every event has two times, the time it happened and the time we received it, and they disagree.

Windowing, and the *two clocks*

Last time we ended with the claim that real streaming questions are questions about time. Before we can group by time, we have to decide which time, because every event in our pipeline carries two of them, and the difference between them is not a technicality. It is the source of most wrong answers in streaming systems.

Event time vs processing time

Event time is when the thing happened in the world: the moment the pitch left the hand, stamped by the tracking system in the stadium. **Processing time** is when our system received or processed the record: the moment the event landed in Kafka, or the moment a micro-batch picked it up. In a perfect pipeline they would differ by a constant few seconds. In a real pipeline the gap is variable and occasionally enormous. The stadium's uplink drops for ninety seconds and then flushes a backlog. A broker rebalance stalls a partition. An upstream service retries a failed send three minutes later. The GitHub Archive feed you consume in lab publishes some events well after they occurred, and Wikipedia Pageviews arrive in hourly files whose contents span the hour unevenly. Every real stream is somewhat out of order with respect to event time, and arrival order is a lie about world order.

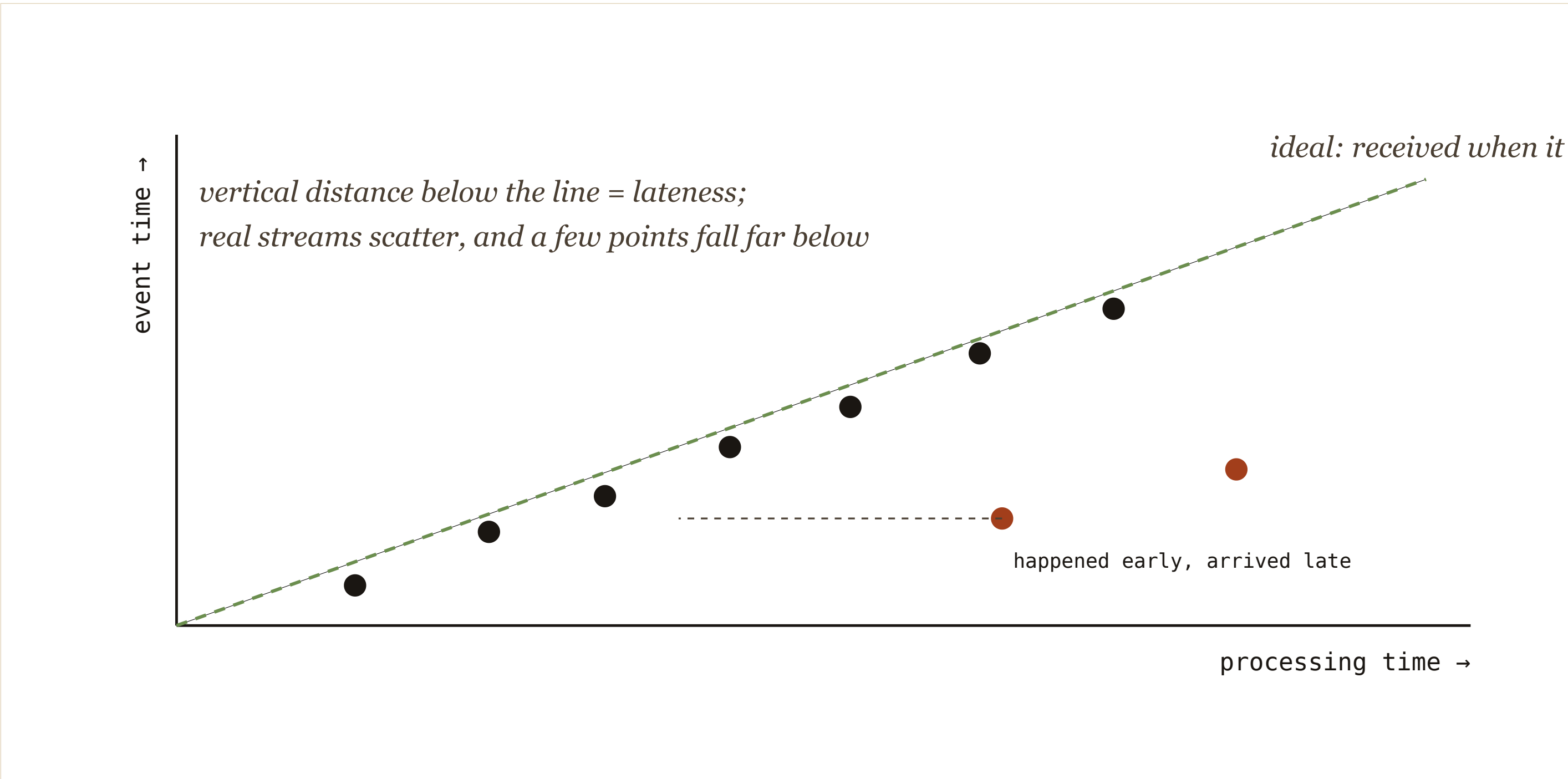

FIG 3.8 *Each dot is one event, plotted by when it happened against when we saw it. The diagonal is the fantasy; the scatter is reality; the deep stragglers are the late data that the rest of this unit is about.*

Which clock should a query group by? Processing time is operationally easy (the engine always knows what time it is) and semantically weak: "pitches received between 19:00 and 19:05" is a fact about our pipeline's plumbing, and the same input replayed tomorrow gives different answers. Event time is semantically right ("pitches thrown between 19:00 and 19:05" is a fact about baseball, stable under replays and outages) and operationally hard, because the stream is out of order in event time and we never know for certain when a time period is fully accounted for. Here's our rule: analytics group by event time, and the operational hardness gets dealt with head-on, which we will tackle later. Today we take the grouping itself.

Windows

A **window** is a time-bounded slice of the stream that turns "aggregate forever" into "aggregate per interval." Grouping by window is what makes streaming aggregations answer real questions, and it is also what bounds their state, since a window, unlike an all-time group, can eventually finish. Three window shapes cover nearly everything.

Tumbling windows tile the timeline edge to edge with fixed-size, non-overlapping intervals. Pageviews per hour, pitches per inning-length slice: each event belongs to exactly one window, and the per-hour counts sum back to the total. Use tumbling windows when the question is "how much per period."

Sliding windows have a fixed size but advance by a smaller step, so consecutive windows overlap and each event belongs to several. A ten-minute window sliding every two minutes gives a fresh reading of "average velocity over the last ten minutes" every two minutes, smoothing the jitter a tumbling sequence would show. The cost is mechanical and predictable: with size 10 and slide 2, every event is bookkept in five windows, so state and write volume are about five times the tumbling equivalent. Use sliding windows when the question is "what is the recent trend," and notice the smoothing is the moving average from your statistics coursework, computed online.

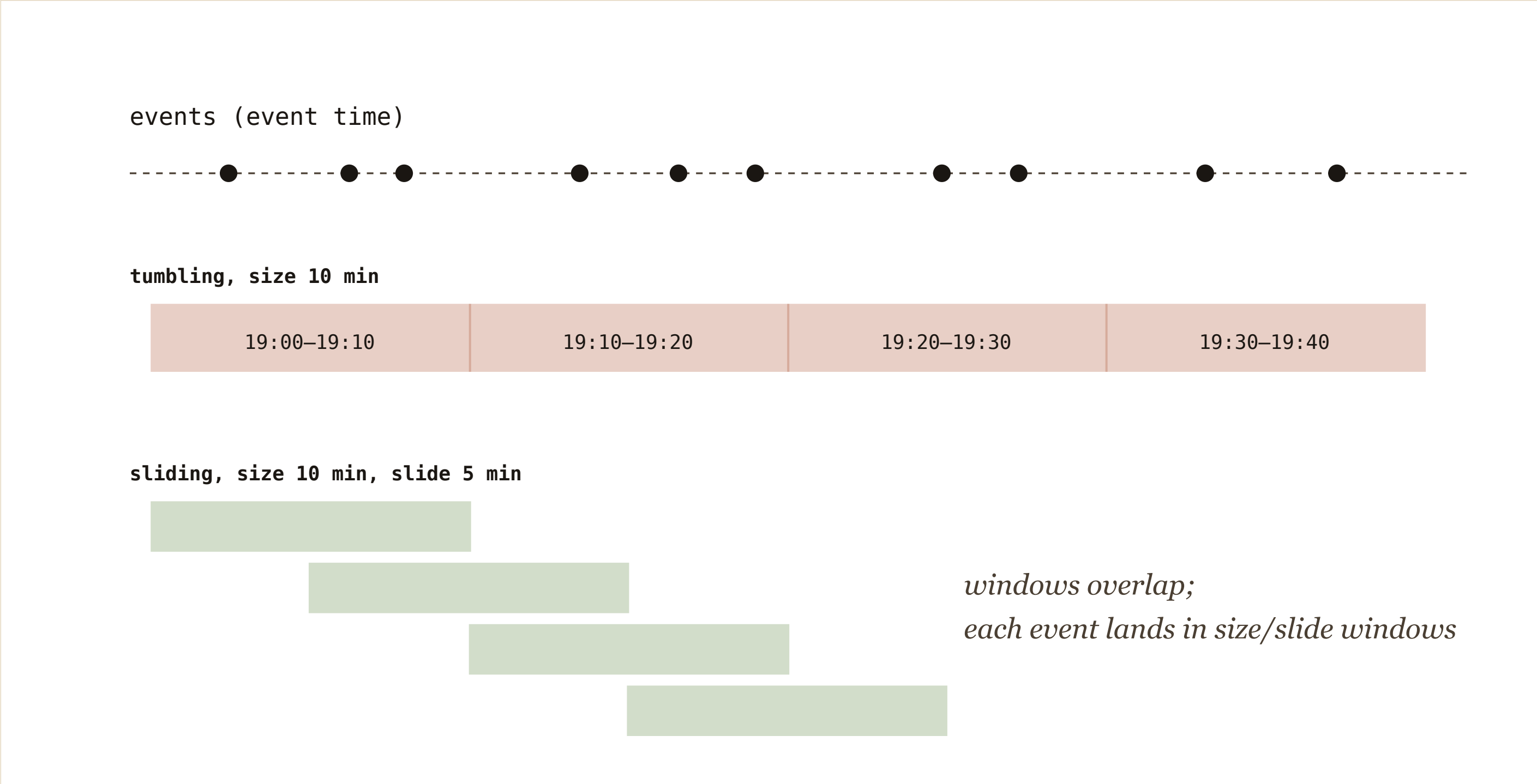

FIG 3.9 *Tumbling windows tile the event-time axis; sliding windows overlap it. The choice is the question's: totals per period want tumbling, recent trends want sliding.*

Session windows abandon the fixed grid entirely. A session groups events that arrive close together in event time and closes when a **gap** of inactivity exceeds a threshold. The window's extent is data-driven: MIDN B's browsing of the team's stats site this evening is one session, ended by the thirty quiet minutes after she closes the laptop, and her session tonight will have a different length than her session tomorrow. Sessions are the natural shape for user activity, and they are also the expensive shape, because the engine must hold every session open in state until its gap elapses, and a new event can extend a session, or even merge two sessions that no longer have a gap between them, retroactively.

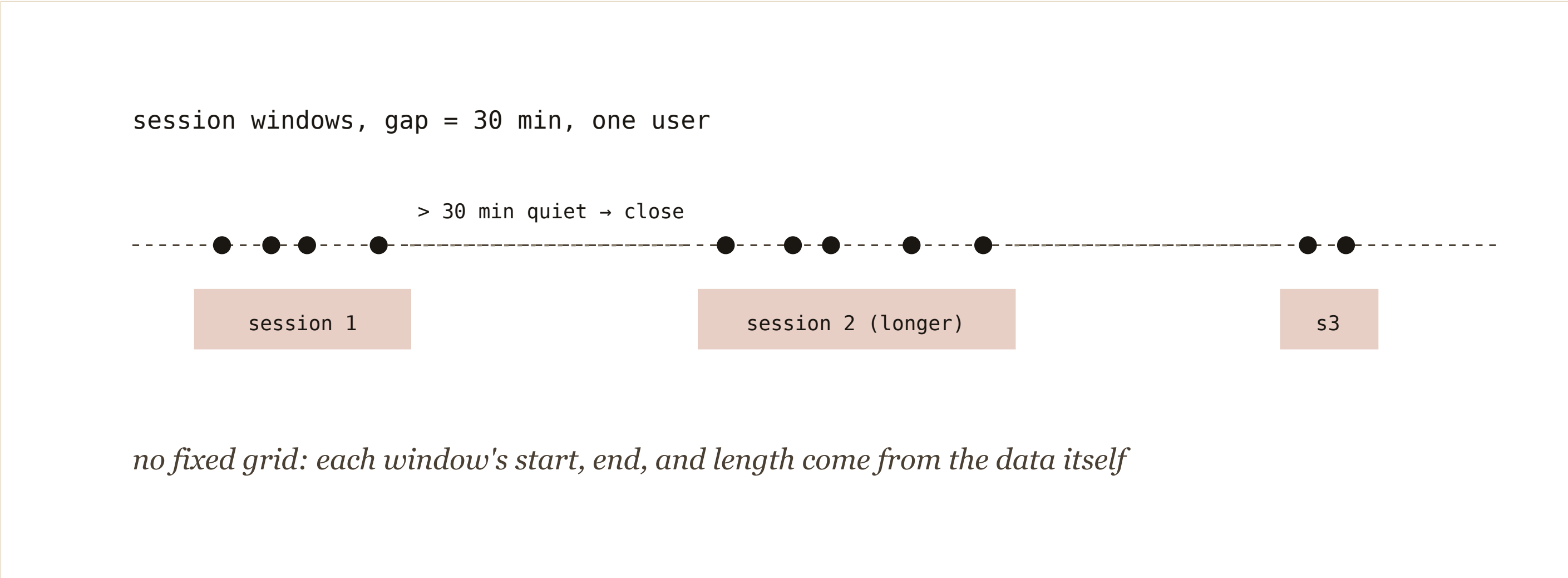
session windows, gap = 30 min, one user
> 30 min quiet → close
session 1
session 2 (longer)
s3
no fixed grid: each window's start, end, and length come from the data itself

FIG 3.10 *Session windows. Activity clusters define the windows; a configured gap of silence ends them. Window boundaries are an output of the computation, not an input.*

Windowing in Structured Streaming

The API expresses all of this as grouping. A window is just another grouping column, built from the event-time column:

```
velocity = (events
    .groupBy(
        window(col("pitch_ts"), "10 minutes", "2 minutes"),   # size, slide
        col("pitcher"))
    .agg(avg("release_speed").alias("avg_velo")))
```

Omit the slide argument and the window tumbles; `session_window(col("pitch_ts"), "30 minutes")` gives sessions. The result table's key is (window, pitcher), and the rows look like ordinary rows with an interval-typed column:

```
+------------------------------------------+---------+----------+
| window                                   | pitcher | avg_velo |
+------------------------------------------+---------+----------+
| {2026-07-14 19:00:00, 2026-07-14 19:10}  | Skubal  |   97.4   |
| {2026-07-14 19:02:00, 2026-07-14 19:12}  | Skubal  |   97.1   |
| {2026-07-14 19:04:00, 2026-07-14 19:14}  | Skubal  |   96.8   |
+------------------------------------------+---------+----------+
```

Each key's row updates as events for that window arrive, late ones included: a straggler whose `pitch_ts` falls in 19:00–19:10 is credited to 19:00–19:10 no matter when it shows up, which is exactly the event-time correctness we wanted. Read those three overlapping rows again and you can see a fatigue signal forming (96.8 and drifting down), which is precisely the between-innings question the coaching staff asked for back on the first day of this unit.

Choosing the window's size is the analyst's half of the design, and it is a resolution decision with the same flavor as bin width in a histogram. Too narrow (a one-minute window on a pitcher who throws fifteen pitches an inning) and most windows hold one or two events, the averages are noise, and state multiplies. Too wide (a three-hour window) and the fatigue signal you wanted is averaged into invisibility. The right size comes from the question and the data density, not from the engine, and the windowing lab has you sweep window sizes over Wikipedia Pageviews and watch a real traffic spike sharpen and dissolve at different resolutions.

Run the velocity query above for a moment in your head, though, and the trap springs. Every (window, pitcher) pair the stream has ever produced is a row of state, and nothing we have written ever lets one go. The 19:00–19:10 windows cannot be declared finished, because how do we know some straggler from 19:07 is not still out there in a stalled stadium uplink? Without an answer, state grows monotonically all season, and append mode (emit a window once, when final) is impossible, because "final" is undefined. What we are missing is a principled way for the system to say: enough time has passed; I am declaring this window complete, accepting that anything later still belonging to it will be dealt with by policy rather than by waiting forever.

Watermarks and *late data*

We left off holding every window open forever, because with event time there is no moment at which we *know* a window has seen its last record. That knowledge is genuinely unavailable; no protocol can conjure it, because the missing straggler is, by definition, somewhere we cannot see. What a system can do instead is replace knowledge with a policy: decide in advance how long to wait, wait that long, and then act as if the window is complete. The mechanism that implements the policy is the watermark.

The watermark contract

A **watermark** is the engine's moving estimate of event-time progress: a threshold T meaning "I am proceeding as though no event with event time earlier than T is still coming." The application declares its patience once:

```
events_wm = events.withWatermark("pitch_ts", "10 minutes")

velo = (events_wm
    .groupBy(window(col("pitch_ts"), "10 minutes"), col("pitcher"))
    .agg(avg("release_speed").alias("avg_velo")))
```

and the engine maintains the threshold mechanically: watermark = the maximum event time observed so far, minus the declared delay. If the latest `pitch_ts` the query has seen is 19:30:00 and the delay is ten minutes, the watermark stands at 19:20:00. As newer events arrive, the maximum advances and the watermark follows, always trailing the leading edge of the stream by the declared allowance. Note carefully what drives it: the data's own timestamps, not the wall clock. If the stream stalls, the watermark stalls with it, which is the correct behavior, because a quiet stream is exactly the situation in which we have no evidence that time has moved on.

The watermark earns its keep twice. First, it defines lateness precisely: an event is **late** when its event time is older than the current watermark, meaning it arrived after the engine, by policy, stopped waiting for it. Events within the allowance are not late in any meaningful sense; they are reordered, the engine credits them to their proper windows, and the results are exactly as if they had arrived on time. Second, and just as important, the watermark is the state janitor we have been missing. Once the watermark passes the end of a window plus the allowance, no acceptable event can ever land in that window again, so the engine can finalize the window's result, emit it (this is what makes append mode legal for aggregations), and evict its state. The all-season state growth becomes a rolling buffer covering roughly the windows the watermark has not yet swept past. State is bounded because waiting is bounded.

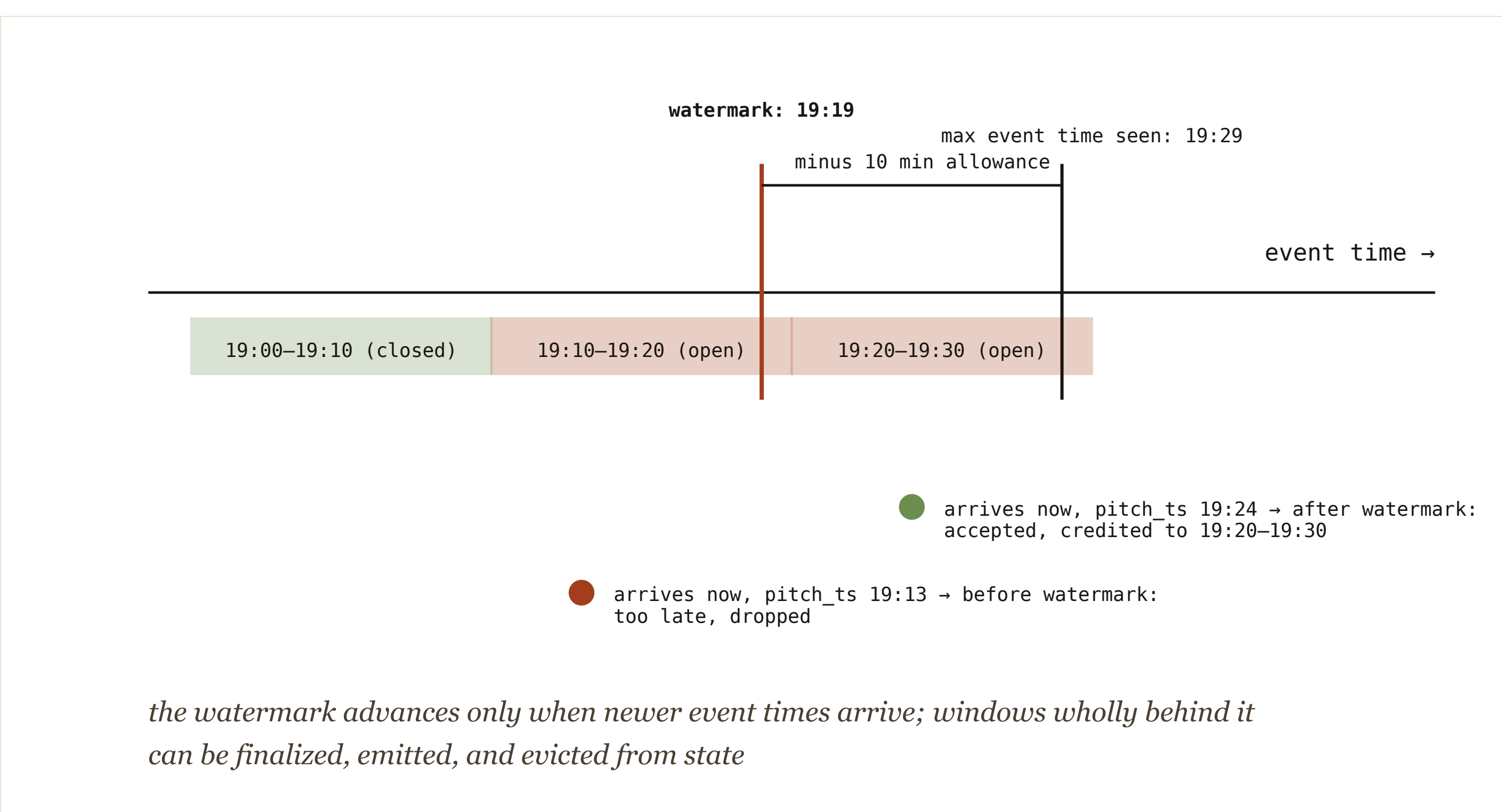
watermark: 19:19
max event time seen: 19:29
minus 10 min allowance
event time →
19:00–19:10 (closed)
19:10–19:20 (open)
19:20–19:30 (open)
arrives now, pitch_ts 19:24 → after watermark:
accepted, credited to 19:20–19:30
arrives now, pitch_ts 19:13 → before watermark:
too late, dropped
the watermark advances only when newer event times arrive; windows wholly behind it
can be finalized, emitted, and evicted from state

FIG 3.11 *The watermark trails the maximum observed event time by the declared allowance. Stragglers ahead of it are quietly absorbed; stragglers behind it are past the policy's patience.*

The watermark, as a formula

watermark = max(event time observed across the query) − declared allowance. Two things follow immediately. The watermark never moves backward, because the max never decreases. And the watermark only moves *forward* when newer event times arrive, so a stream that goes quiet freezes its own watermark in place, holding every open window open until data resumes.

That second property has a sharp operational edge in a partitioned world, and it bites real deployments often enough to flag in class. The query's watermark must be safe with respect to *every* Kafka partition it reads, so it advances with the slowest partition, taking the minimum across them. Now suppose the topic has sixteen partitions and one of them goes idle (the East Coast games ended; that key range has no more traffic today). The idle partition reports no new event times, the minimum stops advancing, and the watermark for the whole query stalls, which means no window anywhere finalizes and state quietly accumulates hwile the rest of the country continues to log events. The fix is configuration, not code (Kafka sources can be told to mark idle partitions as such after a timeout), but you only reach for the fix if you recognize the symptom, and the symptom is exactly what the streaming UI's watermark metric is for. When finalized output stops flowing and state rows climb, check whether the watermark is moving before you suspect anything else.

Latency vs completeness

The delay parameter is a tradeoff with two ends and no free middle. Set the allowance long (an hour) and almost every straggler is absorbed, so results are nearly complete; but windows take an extra hour to finalize, append-mode output runs an hour behind the world, and an hour's worth of open windows sits in state. Set it short (ten seconds) and results are fast and state is lean, but every uplink hiccup sheds events, and the per-hour counts quietly undercount. There is no correct value in the abstract. There is a correct value *for a measured stream and a stated requirement*, which makes this a build-then-measure decision like every other one in this course: take a day of the real feed, compute arrival time minus event time for every record, and look at the distribution's tail. If 99.9 percent of GitHub Archive events arrive within four minutes of their event time, a five-minute watermark buys you three-nines completeness at five minutes of result latency, and you can write that sentence, with the histogram behind it, in a design document. In lab you will measure exactly this distribution and defend your chosen allowance with it.

What happens to the truly late

And the 0.1 percent behind the watermark? In Structured Streaming's windowed aggregations they are dropped, silently, by design: accepting them would mean reopening finalized windows and retracting emitted results, and the engine refuses that complexity. "Silently" should make you slightly uncomfortable, and the remedy is to make the loss visible and deliberate rather than invisible and accidental. Monitor the drop (Spark exposes late-row counts in the streaming query's progress metrics; the lab plots them). If the dropped tail matters to the business, route it: a second consumer reading the same Kafka topic (replayability, again) can divert too-late records to a corrections table that a nightly batch job folds into the gold copy. The streaming answer is fast and 99.9 percent complete; the batch answer is slow and exact; the architecture that runs both and reconciles them is common enough to have a name in industry, and you now know precisely which problem each half solves.

Step back and look at what we have built across the last four classes. Events flow through Kafka, Spark consumes them incrementally, windows organize them by the time they happened, watermarks bound both the waiting and the state. The pipeline is correct while it runs. The phrase doing the quiet work in that sentence is "while it runs," and a pipeline that runs for a season does not get to assume it. Executors die mid-batch. The driver is killed mid-commit by a deploy. The sink accepts half a write and times out. Next, we will ask the question the whole unit has been building toward: when the crash comes with events in flight, what, exactly, does the output contain afterward, and what would it take to make the answer "exactly what it should"?

Exactly-once and state stores

Here is the question we have been building toward, in its sharpest form. The pipeline crashes at 21:43, mid-batch, with ten thousand events somewhere between Kafka and the sink. We restart it. For each of those ten thousand events, exactly one of three things is now true of the output: the event's effect is missing, present once, or present more than once. Which one, and can we choose?

The three semantics

The possible promises a pipeline can make have standard names. **At-most-once**: every event affects the output zero or one times; failures lose data, never duplicate it. This is what you get by committing offsets *before* processing: the crash happens after the commit, the restart resumes past the unprocessed events, and they are simply gone. Cheap, fast, and fine for telemetry where a gap is tolerable. **At-least-once**: every event affects the output one or more times; failures duplicate data, never lose it. This is what you get by committing offsets *after* processing: the crash happens after the sink write but before the commit, the restart replays from the old offset, and the events are written again. **Exactly-once**: every event affects the output precisely once, crash or no crash. This is the promise we actually want for counts, sums, and anything feeding a decision, because a duplicated aggregate is not "slightly stale," it is wrong.

Notice that the failure window producing duplicates is structural, not a bug. Processing the batch and recording that the batch was processed are two operations, a crash can land between any two operations, and no amount of careful ordering removes the gap; reordering only converts loss into duplication or back. SD321 gave us the right lens for this: it is an atomicity problem. Several effects (read these offsets, update this state, write this output) must become visible all together or not at all. Exactly-once streaming is the A in ACID, rebuilt across a message bus, a compute cluster, and a sink that may be a pile of Parquet files.

What "exactly-once" actually means

One honest clarification before the recipe, because the term oversells. No system can guarantee each event is *processed* exactly once; a crashed batch will be reattempted, and the same record will flow through the same code twice. What can be guaranteed is that the *observable effect* is as if it were processed once: reprocessing happens, but its results replace rather than add. The honest term is effectively-once, and the engineering content is a recipe with three legs, each leg owned by a different component of our pipeline.

1. **A replayable source.** Recovery requires re-reading the exact records of the failed attempt. Kafka's log gives us this for free: the checkpoint names an offset interval, and offsets 4,107 to 4,933 are the same bytes today as yesterday, as long as retention has not swept them. This is the moment the read-without-delete decision from earlier pays off completely; a transport that deleted on delivery would make recovery impossible at the first leg.
2. **Versioned state, committed atomically with progress.** The checkpoint must record "offsets through 4,933" and "the state after applying them" as one indivisible fact. Structured Streaming versions state by batch id and commits the batch's offsets and state delta together, so a restart loads a consistent pair: either the batch fully happened or it fully didn't.
3. **A sink that tolerates retried writes.** The replayed batch will attempt its output again, so the sink must make the second attempt a no-op. Either the write is **idempotent**, meaning applying it twice leaves the same result as once (overwriting a keyed row with the same value; writing a file under a deterministic name derived from the batch id, which is how the Parquet file sink works), or the sink is transactional and the write commits atomically with a batch id it can use to recognize and skip a duplicate. A sink that blindly appends, like a plain INSERT, caps the whole pipeline at at-least-once no matter how careful everything upstream is.

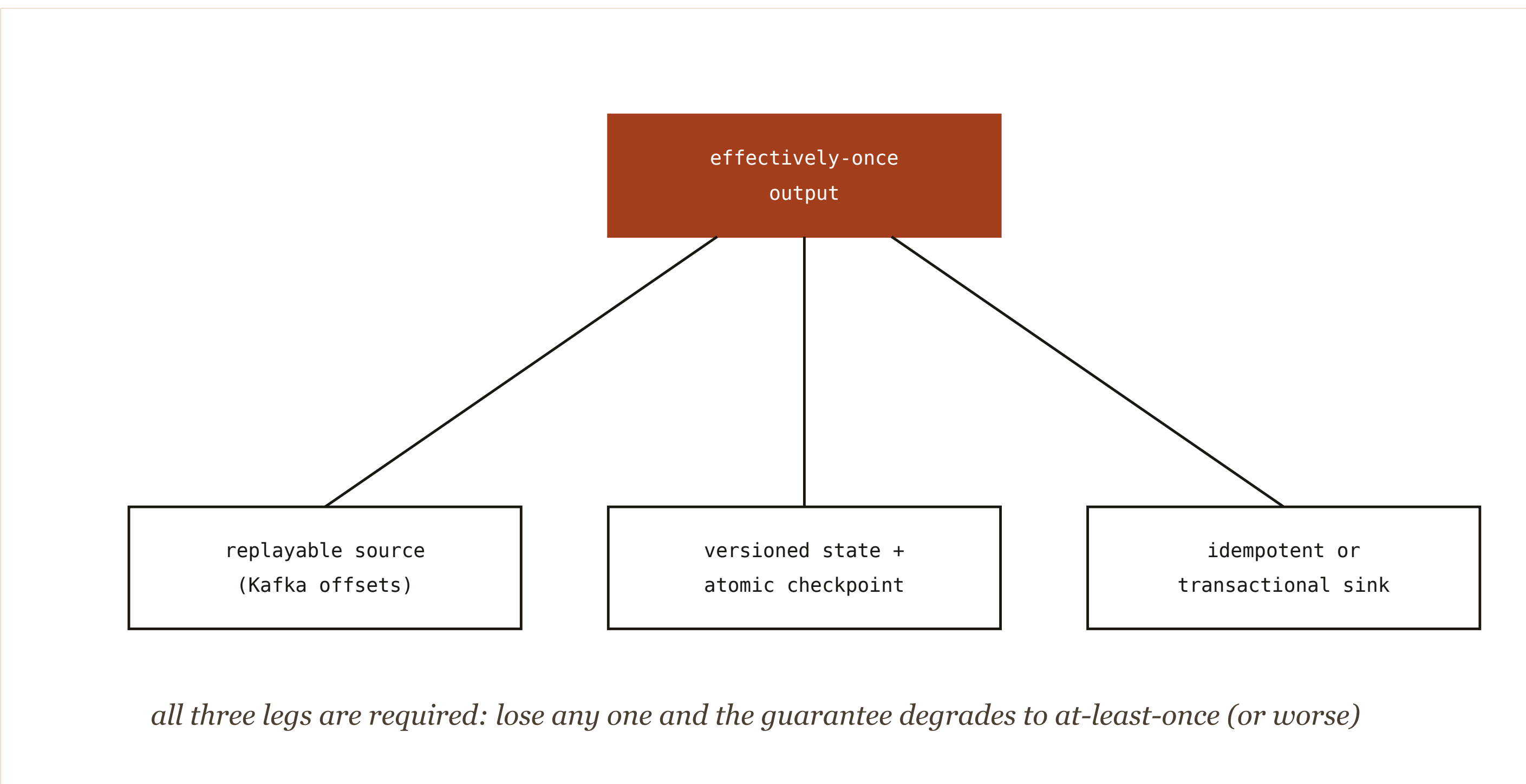

FIG 3.12 *The effectively-once recipe. Replay re-reads the failed batch, the checkpoint restores a consistent offsets-and-state pair, and the sink absorbs the retried write without double-counting.*

The recipe also tells you how to audit a pipeline someone hands you, which is a capstone skill. Walk the legs. Is the source replayable, and is retention longer than the worst outage you intend to survive? Are offsets and state committed together, in a checkpoint that survives the cluster? Does the sink recognize a retried batch? The weakest leg is the pipeline's real guarantee, and the most common honest answer in industry is "exactly-once into the lake, at-least-once into the dashboard," which is often exactly the right engineering call, since a dashboard refresh self-corrects and a lake does not.

For sinks that Spark does not handle natively (a Postgres table, an external API), the practical tool is `foreachBatch`, which hands your code each micro-batch as an ordinary bounded DataFrame together with its batch id. Your code then owns leg three. The clean pattern is to make the batch id do the work: write the batch and record its id in the same database transaction, and on a retry, check for the id first and skip if present. SD321 again, almost embarrassingly directly; the streaming engine has reduced its hardest guarantee to "please do one ACID transaction per batch," and your databases course taught you how to do that.

Lets also be honest about the bill, since everything has a price. Exactly-once costs a per-batch checkpoint write (offsets plus state delta, durably, to object storage in our stack), which puts a floor under the batch interval; push triggers much below a second and the commit overhead starts to dominate the useful work. It constrains your choice of sinks to the idempotent and the transactional. And it adds recovery time after a failure, since the engine must reload state and replay the in-flight interval before fresh output resumes. Every one of those costs is usually worth paying for data feeding decisions, and usually not worth paying for a metrics feed that refreshes itself every thirty seconds anyway. Choosing which guarantee each output of a pipeline actually needs, and saying so out loud in the design, is the mark of someone who understood this.

State stores

Leg two has been carrying a lot of weight, so lets finally look at where state physically lives. A **state store** is a per-partition key-value store on each executor that holds the streaming query's working memory: window aggregates keyed by (window, key), open sessions, seen-key sets for deduplication. State is partitioned by grouping key with the same hash partitioning as every shuffle, so the executor that processes a key's records is the executor that holds that key's state, and no network hop intervenes in the update path.

The default store keeps state as in-memory hash maps backed by checkpoint files, which is fine until state is large, and "large" is exactly where real workloads live (millions of open sessions; a long watermark allowance times a hot stream). Production deployments switch the store to **RocksDB**, an embedded on-disk key-value engine, with one configuration line, moving state out of the JVM heap and off the garbage collector's books. Each micro-batch commits a new state version tied to its batch id, retaining enough history to rewind a failed batch; the Spark UI's streaming tab shows state rows, state memory, and per-batch commit times, and reading that tab is to a streaming job what reading the stages page was to a batch job. The forensics instinct transfers whole: when a streaming query's batch durations creep upward week over week, the first suspect is state growth, and the first question is: what bounds it?

That closes the loop we opened. The crash at 21:43 now has a precise answer: the restarted query reloads the last committed offsets-and-state pair, replays the in-flight interval from Kafka, recomputes deterministically, and the sink absorbs the retry; the output contains each event's effect exactly once, and we can defend every clause of that sentence by pointing at a mechanism. Lab will make you earn it: you will run a stateful aggregation, kill it mid-batch on purpose, restart it, and prove from the output and the checkpoint that nothing was lost and nothing was double-counted.